Minimizing Vendor Dependence in Container Platforms

27 January 2026 - ID G00841999 - 35 min read

By: Lucas Albuquerque

Initiatives: Technologies and Markets

Contrary to common belief, vendor neutrality is not a guaranteed outcome of container adoption. This note explores how the adoption of open-standard infrastructure and runtime components can help infrastructure architects prevent vendor lock-in and promote digital sovereignty using containers.

Overview

Key Findings

- A growing desire for digital sovereignty is driving organizations to adopt containers and K8s. While many practitioners think that containerization and open-source platforms guarantee portability and freedom from vendor control, true independence requires careful architectural planning to avoid getting locked.
- Proprietary or tightly coupled technologies and managed services in container platforms can lead to vendor dependence. Examples include orchestration engines, computing functions, networking plugins, storage solutions, and runtime components.
- Vendor lock-in is not inherently bad. Pursuing complete vendor independence can lead to unnecessary complexity and higher costs. Using managed container services and proprietary integrations strategically can be a practical and justified approach.
- In some cases, it's more practical to focus team skills on proprietary technologies rather than striving for complete vendor independence, especially when extensive portability is not a top strategic priority and operational simplicity is key.

This research note is restricted to the personal use of .

Recommendations

- Adopt a deliberate, standards-first approach, prioritizing open industry specifications like OCI, CRI, CNI, CSI, OTEL, and others for container platforms over proprietary integrations. This helps manage vendor dependence, ensures workload portability, and achieves digital sovereignty.
- Choose components backed by open-source technologies to reduce reliance on individual vendors' proprietary solutions. The ecosystem minimizes open-source risks, as seen in K8s' extensibility and broad vendor support, thereby reducing vendor dependence.
- Acknowledge that some vendor-specific technologies can be beneficial. Manage them by weighing operational benefits and lower engineering costs against portability and vendor lock-in trade-offs.
- Set your priorities: invest in developing vendor-neutral skills for independence or focus on proprietary knowledge for operational efficiency. Recognize that vendor portability is justified only when portability is a key strategic objective.

Problem Statement

Despite the widespread adoption of and container orchestration platforms such as AWS EKS, Azure AKS, Red Hat OpenShift, SUSE Rancher, Broadcom VMware Tanzu Platform, and others, organizations still face significant challenges with proprietary environments and stack dependencies. A common misconception among practitioners is that adopting containers and automatically leads to vendor independence. However, while containers package applications and dependencies into standardized images, they can unintentionally hide vendor-specific components such as proprietary orchestrators, networking capabilities, or runtime elements, thereby increasing vendor dependence.

Default configurations, managed services, and vendor-specific extensions often introduce subtle forms of lock-in. As a result, adopting containers alone does not automatically achieve infrastructure neutrality, leading to poor cross-platform compatibility and binding workloads to specific vendor ecosystems. This vendor lock-in makes it difficult to migrate workloads and hinders the implementation of hybrid and strategies, even with containerization.

However, this issue is particularly acute in scenarios where portability and flexibility are critical (see Figure 1), such as:

This research note is restricted to the personal use of .

- **Digital sovereignty:** Vendor lock-in undermines digital sovereignty by restricting organizations' control over their data, infrastructure, and technology choices. Reliance on proprietary platforms limits flexibility, increases switching costs, and exposes entities to external risks. This dependency hampers innovation, reduces negotiation power, and challenges compliance with local regulations.

- **Multicloud and hybrid deployments:** Even with containerization, cross-cloud deployments encounter substantial technical obstacles to portability. Applications often depend on provider-specific services, such as proprietary identity and (IAM), infrastructure nuances (e.g., unique node types), storage offerings, and ingress configurations. These dependencies collectively increase switching costs and operational complexity.

- **-based and distributed applications:** Deploying microservices and distributed systems frequently results in entanglement with commercial platform-as-a-service (PaaS) or container-as-a-service (CaaS) solutions. Although containers standardize application packaging, these platforms bundle proprietary toolchains and runtime, attaching workloads to the vendors' integrated environments (see Use Cloud-Native Architecture to Modernize Your Applications).

- **AI workload support:** Achieving true vendor neutrality for AI workloads remains difficult, as performance optimization, such as advanced scheduling, GPU resource sharing, and access to specialized AI accelerators, requires proprietary integrations, vendor-specific device plugins, or managed services. These dependencies introduce lock-in risks and elevate operational complexity (see Optimize AI Workloads With Containers and Kubernetes).

- **scenarios:** Despite leveraging portable containers, edge computing architectures are prone to vendor lock-in due to the complexities of orchestrating and securing numerous remote clusters. Teams often adopt a single vendor's integrated stack or control plane management system to ensure consistent operations and robust security, inadvertently limiting flexibility (see Deploying Containers and Kubernetes at the Edge).

- **Containerized commercial off-the-shelf (COTS) software:** Vendor lock-in constrains the portability of containerized COTS applications. Independent software vendors (ISVs) typically certify their solutions only for specific Kubernetes distributions or cloud platforms. Such requirements restrict deployment options and necessitate reliance on vendors that provide specialized compatibility and support (see Best Practices for Container Release and Life Cycle Management).

This research note is restricted to the personal use of .

Assuming containers and Kubernetes guarantee vendor independence leads to hidden lock-in, as proprietary dependencies limit portability. Achieving true neutrality, especially for sovereignty, multicloud, AI, and edge use cases, requires open standards and intentional infrastructure standardization.

Figure 1: Key Challenges of Vendor Lock-In Across Modern IT Architectures

Key Challenges of Vendor Lock-in Across Modern IT Architectures

Digital Sovereignty	Vendor lock-in restricts digital sovereignty, limiting control and flexibility.
Multicloud and Hybrid Deployments	Multicloud and hybrid deployments face technical portability barriers due to provider-specific dependencies.
Microservices and Distributed Apps	Microservices architectures often become tied to proprietary PaaS/CaaS toolchains and Runtime.
AI Workload Support	AI workloads require vendor-specific integrations, increasing lock-in and complexity.
Edge Computing Scenarios	Edge computing relies on integrated stacks for security and management, reducing flexibility.
Containerized COTS Software	Containerized COTS software depends on certified platforms, constraining portability and deployment options.

Source: Gartner
841999_C

Gartner

This note helps infrastructure architects understand how adopting open standards, leveraging open source initiatives, and making deliberate architectural decisions promote vendor neutrality and digital sovereignty when deploying container platforms. However, it also underlines that efforts to reduce vendor dependence and promote neutrality often add complexity. Therefore, the investment and effort needed to achieve true vendor neutrality must be balanced against the added complexity it brings.

This research note is restricted to the personal use of .

The Gartner Approach

Proprietary technologies and stack dependencies incorporate critical components across both the container infrastructure, such as runtime, storage, and networking plugins, also encompassing integrations for identity, secrets, monitoring, and . These factors are the fundamental drivers of vendor lock-in in container platforms. This tight coupling often results in poor cross-platform compatibility, binding application workloads to specific vendor ecosystems and challenging migration or multicloud strategies.

While proprietary implementations, extensions and technologies may offer convenience, practitioners must prioritize open standards and open source tools within container platforms, whenever they seek for vendor independence. These initiatives foster vendor neutrality by delivering consistent interfaces and interoperable components, reducing reliance on proprietary solutions and simplifying migration across diverse cloud and infrastructure environments.

Quick Answer

How can a container platform be operated while minimizing reliance on specific vendors?

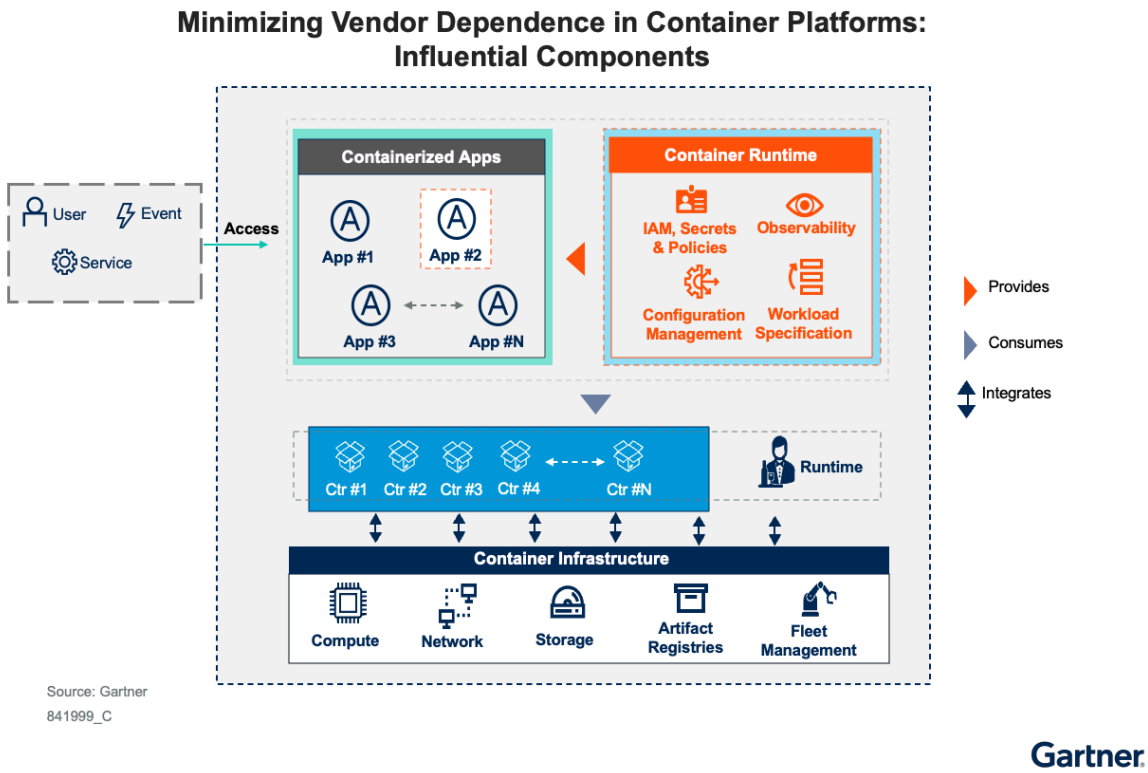
To minimize reliance on specific vendors, practitioners must focus on the following key actions and architectural strategies:

- Prioritize and adhere strictly to open industry specifications (e.g., OCI, CRI, CNI, CSI, etc.) across all platform layers to prevent vendor lock-in proactively.
- Implement deliberate architectural decisions to standardize the infrastructure layer using open standards.
- Prevent reliance on unique cloud provider services and runtime components.
- Utilize the inherent extensibility and open-source nature of platforms like Kubernetes.
- Focus on managing vendor dependence by carefully weighing operational benefits against the strategic costs of decreased portability and lock-in risks.

This research note is restricted to the personal use of .

To minimize vendor dependency in container platforms, Gartner recommends that practitioners pay attention to infrastructure and runtime components that directly influence vendor lock-in. Figure 2 highlights that both infrastructure and runtime layers contain critical elements where vendor lock-in risks may arise. By watching out for proprietary dependencies in these areas, organizations can better strategize for vendor neutrality and portability in their container platforms.

Figure 2: Minimizing Vendor Dependence in Container Platforms: Influential Components



The scope of this note deliberately excludes application portability considerations related to application design principles, as these are typically beyond the purview of infrastructure architects. Instead, the focus is exclusively on infrastructure and runtime decisions that minimize vendor reliance.

The Guidance Framework

Gartner proposes this framework for guiding practitioners through the complex process of achieving infrastructure architecture neutrality in container platforms, where vendor dependence arises from multiple layers. The framework helps identify areas where proprietary technologies often create lock-in by categorizing technology domains into infrastructure and runtime concerns. The framework will address the following domains:

This research note is restricted to the personal use of .

- The Vendor-Neutrality Zone: By briefly examining the current container platform landscape, the prework section will propose a “vendor-neutrality zone” that highlights approaches that favor those pursuing reduced vendor lock-in
- Infrastructure Components: By prioritizing open standards and interoperability across these infrastructure components, organizations can significantly reduce vendor dependence and enhance the flexibility of their container platforms.
- Runtime Components: By focusing on these runtime components and adopting open standards and interoperable tools, organizations can significantly reduce vendor lock-in, enhance flexibility, and support a vendor-neutral container ecosystem.

Mapping these domains enables a standards-first approach, encouraging open specifications like OCI, CNI, CSI, and OTEL over proprietary solutions. This is crucial, as true vendor neutrality requires deliberate standardization and architectural decisions, not just the adoption of containers or Kubernetes. Addressing vendor dependence is essential to unlock the full benefits of containerization and cloud-native architectures across diverse and evolving IT environments.

The Vendor-Neutrality Zone

The landscape of container platforms encompasses a broad spectrum of solutions, each with distinct levels of vendor dependence and operational complexity. Some platforms are tightly integrated with specific cloud providers or proprietary technologies, resulting in greater vendor lock-in but potentially streamlined operations. Others are built on open standards and modular architectures, enabling greater portability and flexibility but often demanding more hands-on management and customization from practitioners.

For a detailed description of the various delivery models available for container platforms, read the Decision Point for Selecting Container Platform Deployment Models.

For example, self-managed K8s distributions generally grant organizations significant autonomy and control over their environments, supporting vendor neutrality through open interfaces and standards. However, this autonomy comes with increased operational complexity, as teams must handle upgrades, scaling, security, and integration tasks. On the other hand, managed and serverless container platforms deliver a superior operations and by abstracting infrastructure management and automating operational tasks. Yet, these conveniences come with proprietary dependencies and reduced flexibility, making it harder to migrate workloads or avoid vendor lock-in.

This research note is restricted to the personal use of .

Figure 3: Not All Container Platforms Are the Same

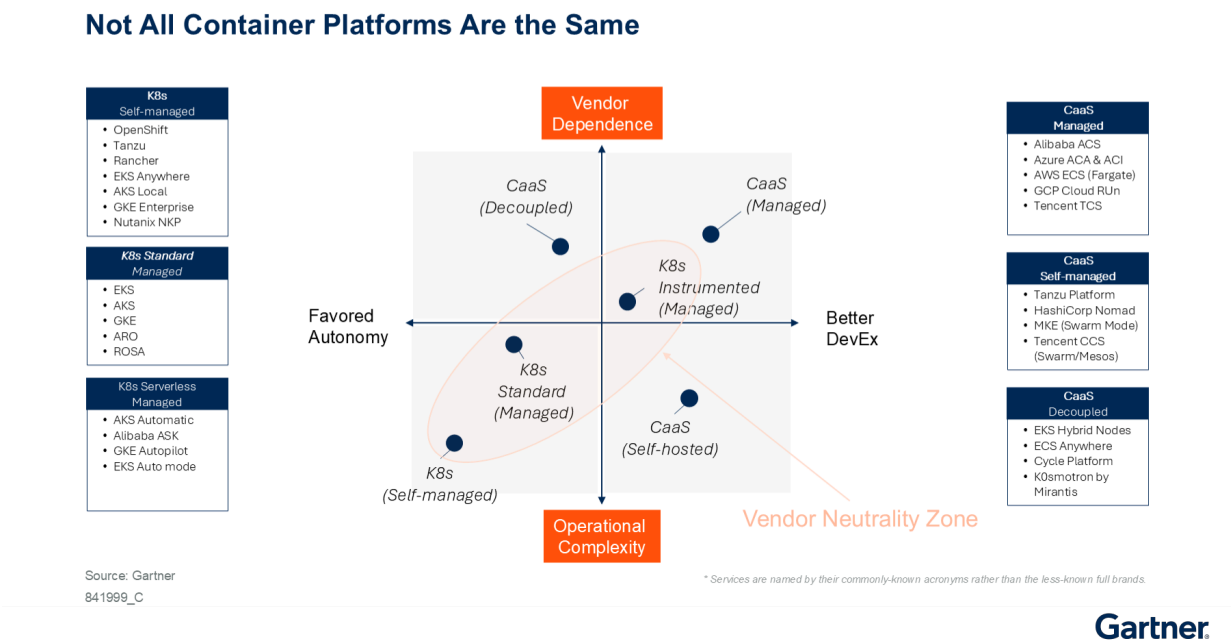


Figure 3 provides a comparative overview of various container platform options, emphasizing differences in vendor dependence, operational complexity, autonomy, and developer experience (DevEx). A lightly shaded area labeled “Vendor Neutrality Zone” highlights platforms that balance operational complexity and vendor neutrality. A “Vendor Neutrality Zone” refers to container platforms that successfully balance the trade-offs between operational simplicity and independence from proprietary vendor ecosystems. These platforms are more prone to accept and support the adoption of open standards, interoperable interfaces, and flexible architectures to reduce vendor lock-in.

The following container platform approaches are part of the “Vendor Neutrality Zone”:

- **K8s self-managed:** This type offers high autonomy and operational complexity with low vendor dependence. Clients must fully manage and maintain their infrastructure, which demands significant expertise and incurs substantial operational overhead (e.g., OpenShift, SUSE Rancher, Mirantis K8s Engine, etc.).
- **K8s standard-managed:** Provides a balance of autonomy and developer experience (DevEx) with moderate operational complexity, as the cloud provider manages the control plane, infrastructure, upgrades, and maintenance, reducing the operational burden while ensuring reliability and security (e.g., EKS, AKS, GKE).

This research note is restricted to the personal use of .

- **K8s serverless-managed:** Delivers a better developer experience (DevEx) but increases vendor dependence by entirely abstracting compute . K8s serverless solutions automatically handle node provisioning, scaling, and maintenance, enabling teams to focus on applications with simplified operations and pod-centric billing (e.g., AKS Automatic, GKE Autopilot).

Practitioners aiming to minimize vendor dependence should focus on solutions within the neutrality zone and choose the option that best meets their requirements for autonomy and manageable operational complexity.

The container landscape presents a trade-off: Managed and serverless solutions offer simplicity and a strong developer experience but risk lock-in. At the same time, self-managed Kubernetes promotes vendor neutrality at the cost of greater complexity.

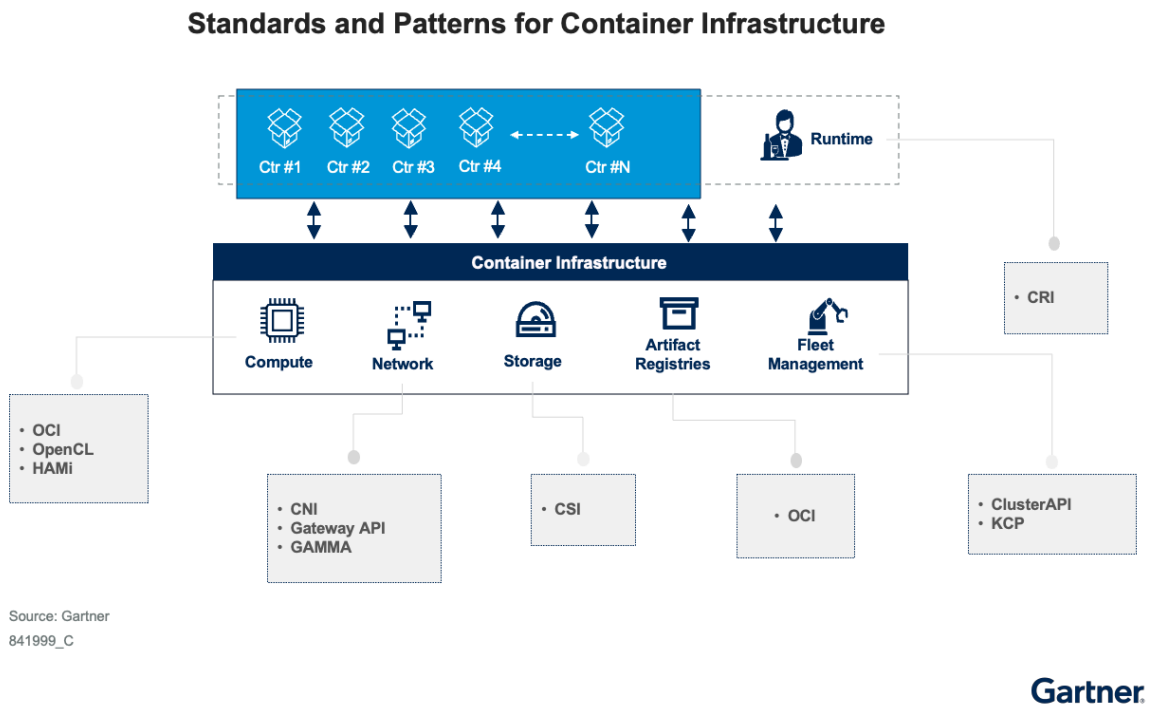
Practitioners should make deliberate architectural choices prioritizing open standards to maintain platform independence and achieve vendor neutrality.

Container Infrastructure

In modern cloud and containerized environments, proprietary technologies and stack dependencies often undermine cross-platform compatibility and inadvertently tie workloads to specific vendor ecosystems. Figure 4 illustrates the essential container infrastructure components that significantly influence the portability and interoperability of containerized workloads across various platforms and vendors. It also highlights the relevant standards, open-source solutions, and community initiatives that enable practitioners to minimize their reliance on proprietary vendor technologies.

This research note is restricted to the personal use of .

Figure 4: Standards and Patterns for Container Infrastructure



1.1. Computing

The computing layer of container infrastructure must go beyond standard CPUs and RAM to support modern AI/ML workloads. These tasks require extensive parallel matrix computations, best handled by GPUs, TPUs, and AI accelerators, which offer thousands of processing units and dramatically speed up operations for applications like large language models (LLMs).

Optimal container infrastructure should support diverse CPU architectures (e.g., x86_64, ARM, PPC) and specialized accelerators to maximize AI/ML workload performance and efficiency. Because accelerators are scarce and costly, techniques like GPU sharing and dynamic resource allocation (DRA) are essential. However, dependence on proprietary, cloud-specific technologies increases vendor lock-in and limits portability.

1.1.1 CPU

The container platform landscape, while still dominated by x86_64, shows a growing availability of ARM in specialized, edge, and hyperscaler workloads. Major vendors offer alternative CPU options, including ARM and PPC options such as AWS Graviton, Google Axion, and IBM PowerPC. Supporting multi-arch images is crucial to ensure application portability across these diverse hardware environments and minimize vendor dependence.

This research note is restricted to the personal use of .

These images are designed to run the same image across different platforms by supporting multiple CPU architectures. Building multi-arch images minimizes reliance on a single architecture, although this may necessitate access to specialized remote services or hardware.

Minimize CPU dependence by:

- **Building multi-architecture images using open tools:** Leverage open-source tools such as Docker BuildX, Kaniko, or Buildah to create container images compatible with multiple CPU architectures (e.g., x86_64, ARM, PPC). These tools enable organizations to support diverse runtime environments and avoid lock-in to a single hardware platform. Building multi-arch images may require access to specialized hardware for effective cross-compilation and testing.
- **Utilizing specialized multi-arch build services:** Adopt cloud-based or managed services, including Depot and Namespace.sh, Docker Build Cloud, and similar platforms that facilitate multi-architecture image building. These services offer advanced features such as build caching, parallelization, and access to specialized hardware resources, streamlining the creation and distribution of multi-arch images (refer to Best Practices for Container Release and Life Cycle Management).
- **Standardizing builds with buildpacks:** Cloud native buildpacks (CNBs) project represents a set of open-source tools and specifications that automate the process of transforming application source code into container images or executable artifacts. Implement buildpacks to automate and standardize container image builds, reducing manual effort and complexity while improving portability, maintainability, and compliance with open standards, and minimizing reliance on proprietary tools.

1.1.2. GPU/TPU/AI Accelerators

NVIDIA's CUDA is the de facto standard for GPU computing due to its robust ecosystem and high performance. Still, its proprietary, closed-source nature significantly hinders vendor neutrality and creates lock-in. Without CUDA, workloads often revert to the CPU, leading to significantly higher latency, slower processing times, and potential system crashes due to memory constraints. This dependence often leads to higher costs and limited hardware choices, particularly because mixed-vendor GPU clusters (such as those combining CUDA and AMD's ROCm) rarely interoperate, resulting in siloed workloads and underutilized resources.

This research note is restricted to the personal use of .

Open standards and vendor-neutral alternatives have emerged to counter this "CUDA lockin," including AMD's ROCm (an open-source platform) and Intel's oneAPI (built on the SYCL open standard). Additionally, runtimes like hami actively promote vendor neutrality by managing and scheduling heterogeneous AI devices from multiple vendors within Kubernetes clusters, thereby improving resource utilization.

Minimize GPU dependence by:

- **Adopting open-source GPU frameworks:** Utilize open-source frameworks such as AMD's ROCm, Intel's oneAPI, and OpenCL to enable cross-vendor GPU compatibility and portability. These frameworks abstract hardware-specific APIs, allowing AI/ML workloads to run across a broader range of GPUs and reducing reliance on a single vendor's ecosystem (see Reference Architecture Brief: AI Infrastructure with Kubernetes).
- **Investing in runtime solutions:** Incorporate runtime layers like HAMi, Volcano, and TensorFusion to further abstract and orchestrate GPU resources within Kubernetes and other container platforms. These solutions facilitate heterogeneous hardware scheduling, dynamic resource sharing, and improved workload management across different GPU types, fostering a more flexible and vendor-neutral infrastructure (see Reference Architecture Brief: AI Infrastructure System Software Stack).
- **Evaluating dynamic resource allocation (DRA):** Assess the implementation of Dynamic Resource Allocation (DRA) frameworks for Kubernetes to achieve vendor-neutral and fine-grained GPU resource allocation. DRA improves GPU utilization and scheduling, supports advanced GPU features, and enhances the efficiency of AI inference and serving workloads in containerized environments (see Adapting Kubernetes for AI Inference and Serving Workloads).

1.2. Networking

Container networking must manage North-South (N-S) and East-West (E-W) traffic. N-S traffic refers to data flowing between containers and external networks, such as user requests entering or leaving the cluster. E-W traffic describes communication between containers or services within the same cluster or data center.

This research note is restricted to the personal use of .

For East-West communication, the container network interface (CNI) is the open standard for container networking in platforms like K8s. CNI plugins are crucial for dynamically configuring network resources, including IP addresses, routes, and policies. Leveraging the K8s native NetworkPolicy API ensures consistent network policies regardless of the underlying CNI vendor, minimizing vendor dependence. However, practitioners must be aware that some CNI plugins offer extended capabilities beyond the open standard, and adopting these specialized features may compromise portability.

Minimize East-West (intra-cluster) network dependence by:

- **Avoiding vendor-specific CNI implementations:** Refrain from adopting CNIs tightly coupled with specific cloud providers or proprietary platforms, such as VMware NSX-T CNI, AWS VPC CNI, and Azure CNI. These solutions can introduce platform lock-in, limit portability, and complicate migration to other environments.
- **Choosing widely adopted, cross-platform CNI alternatives:** Opt for industry-standard, open-source CNIs like Cilium and Calico, which are broadly supported across multiple infrastructures and cloud providers. These CNIs offer robust networking, security, and observability features while maintaining high portability and reducing the risk of vendor lock-in.
- **Prioritizing kubernetes-native networking features:** Favor the use of features and APIs defined in Kubernetes-native specifications, such as the NetworkPolicy API, for network segmentation, security, and traffic control. Relying on standardized K8s constructs ensures consistent behavior across environments and maximizes interoperability, regardless of the underlying CNI implementation.

The Gateway API is a vendor-neutral, extensible standard that provides advanced capabilities for North-South traffic routing and gateway management. Numerous vendors, including major hyperscalers, implement its specifications. This approach is part of the strategy to rigorously standardize the infrastructure layer using open standards (such as CNI and GAMMA) to reduce reliance on proprietary solutions and vendor lock-in in container networking. Initially focused solely on ingress traffic, the Gateway API's specifications were expanded in 2022 through the GAMMA (Gateway API for Mesh Management and Administration) initiative to encompass meshing technologies.

Minimize North-South traffic (ingress/egress communication) dependence by:

This research note is restricted to the personal use of .

- **Ensuring compliance with the gateway API standard:** Ensure all North-South traffic components (e.g., load balancers, proxies) comply with the vendor-neutral Gateway API standard, which enables advanced ingress routing and is widely supported by leading vendors. Adopting this standard supports rigorous infrastructure standardization using open standards (e.g., OCI, CNI, CSI), reduces reliance on proprietary features, and streamlines migration across environments.

- **Integrating specialized** : Compliance must explicitly extend to AI Gateways. These gateways function as LLM proxies, implementing critical features such as rate limiting, authentication, load balancing, and observability. Furthermore, leveraging the K8s official Gateway API inference extension introduces inference-optimized features such as model-aware routing, serving priority, model rollouts, and customizable load balancing.

- **Rationally assessing proprietary solutions:** Teams must thoroughly ponder using proprietary solutions (such as AWS App Mesh, AWS VPC Lattice, or GCP Cloud) that offer seamless integration with the underlying vendor stack. While proprietary dependencies can lead to poor cross-platform compatibility and complicate multicloud or migration strategies, reliance on them is strategically justified when the access to specialized features and integrated support they offer outweighs the benefits of platform neutrality.

- **Bridging the gap between N-S and E-S:** Leverage the GAMMA initiative to enhance the Kubernetes Gateway API to cover service mesh requirements. GAMMA provides a standardized, vendor-neutral approach to managing both North-South and East-West traffic. It extends this standardization to both North-South and East-West (interservice) communications. Key GAMMA-conformant meshing technologies currently include production-ready options such as Google Cloud Service Mesh (GA), Istio (GA), Kuma (GA), and Linkerd (GA).

1.3. Storage

This research note is restricted to the personal use of .

Neutrality for container storage is challenging due to the diverse access methods and requirements, spanning Block, Files, and . This complexity is further heightened by the diverse performance and storage characteristics offered across cloud providers and on-premises environments. The CSI Plugin functions as a vital interface specification, enabling container platforms to interact with external storage systems in a standardized, vendor-neutral way. CSI drivers specifically provision, attach, mount, and manage storage volumes by translating API requests into actions on the underlying storage system. While implementing open standards like CSI is key to mitigating vendor dependence, practitioners should note that vendors exhibit varying levels of maturity and feature sets for container support.

Minimize storage dependence by:

- **Using object storage and CDN solutions**: To minimize reliance on direct storage, avoid attaching persistent volumes to containers whenever possible. Instead, object storage and content delivery network (CDN) solutions can be utilized for scalable, high-performance content distribution, reducing operational complexity and improving portability across environments.
- **Selecting storage solutions with native CSI drivers:** When persistent storage is required, choose storage appliances or (SDS) platforms that provide native, vendor-supported open-source CSI drivers (e.g., Ceph/Rook, Longhorn). Ensure these solutions support essential capabilities, including life cycle management (LCM), multiple access modes, dynamic provisioning, and snapshots, to maintain operational flexibility and compatibility with container orchestration platforms.

For requirements that demand advanced features, such as cross-site replication, high availability, or disaster recovery, opt for commercial storage products (e.g., NetApp, Pure Storage Portworx). These offerings deliver enterprise-grade capabilities, robust support, and enhanced integration with containerized workloads, ensuring reliable performance and simplified management in complex, multisite environments.

1.4. Container Runtime

This research note is restricted to the personal use of .

A container runtime manages the life cycle of container workloads in accordance with the OCI specifications. Popular runtimes include containerd, CRI-O, Docker, and others like MCR and Kuasar. The container runtime interface (CRI) is a key API specification enabling Kubernetes (K8s) to communicate with and manage different container runtimes in a consistent, pluggable, and vendor-neutral manner. This adherence to open standards, such as CRI and OCI, is crucial for rigorously standardizing the infrastructure layer, reducing reliance on proprietary features, and facilitating easier migration. However, runtime-specific features and extensions must be considered. Different workloads may require specific extensions, such as the NVIDIA plugin for containerd, gisor, Firecracker, Snapshots, or NRI, to define their workload specifications effectively.

Minimize container runtime dependence by:

- **Mandating OCI Compliance for Image Portability:** Ensure all container images comply with the OCI standard, which defines specifications for container formats, runtimes, and registries. OCI compliance reduces reliance on proprietary features and simplifies migration across vendors and environments. This portability also extends to non-code assets, as OCI specifications now cover artifacts such as AI models, parameters, and templates, ensuring broad compatibility.
- **Avoiding Proprietary Runtimes and Extensions:** Practitioners should avoid proprietary runtimes and extensions not widely supported across platforms, as these dependencies increase vendor lock-in and hinder portability. While runtime-specific features (e.g., NVIDIA plugin for containerd, gVisor, Firecracker, Snapshots, NRI [Node Resource Interface]) offer specialized capabilities, relying on them can limit cross-platform compatibility and complicate multicloud or migration efforts.
- **Prioritizing the Vendor-Neutral CRI Standard:** Leverage the CRI standard as the primary mechanism for runtime management. CRI is an API specification that enables Kubernetes (K8s) to communicate with and manage different container runtimes in a consistent, pluggable, and vendor-neutral manner. This approach to managing runtimes helps adhere to a standards-first policy, ensuring compatibility and abstraction across various implementations, including popular runtimes like containerd, CRI-O, and Docker.

This research note is restricted to the personal use of .

- **Opting for Containerd for Industry Coverage:** Teams are advised to opt for Containerd due to its broader industry and community coverage. As one of the most popular runtimes, choosing containerd aligns with the general strategy of favoring components backed by open-source technologies. This selection contributes to the effort to rigorously standardize the infrastructure layer using open standards, mitigating the risk of vendor lock-in associated with less widely adopted or proprietary solutions.

1.5. Artifact Registries

Using artifact registries allows teams to maintain private repositories and implement automated workflows for efficient image delivery to execution platforms. Modern artifact registries are essential for managing container infrastructure and promoting vendor neutrality. These registries, which include solutions such as Docker Hub, ECR, ACR, Harbor, JFrog, Quay, etc., provide crucial support for OCI artifacts and can typically handle the storage and distribution of container images across diverse infrastructures. They often incorporate capabilities for replication and syncing to ensure availability. By ensuring that container registries are OCI-conformant, teams can adhere to open standards that reduce reliance on proprietary features and facilitate easier migration between environments.

Minimize artifact distribution dependence by:

- **Opting for vendor-native registry services**: Adopt a managed-first approach for container registries, always prioritizing proximity to the execution platform. Table 1 lists OCI-conformant registry platforms and services.

This research note is restricted to the personal use of .

Table 1: Container Registry Alternatives

Managed Container Registries	Self-hosted Container Registries
▪ Amazon Elastic Container Registry (ECR) ▪ Google Artifact Registry (formerly Google Container Registry, GCR) ▪ Azure Container Registry (ACR) ▪ GitHub Container Registry ▪ GitLab Container Registry ▪ Quay.io ▪ Docker Hub ▪ Oracle Cloud Container Registry ▪ IBM Cloud Container Registry	▪ Harbor ▪ Docker Distribution (Docker Registry) ▪ Quay ▪ Nexus Repository OSS ▪ GitLab Container Registry ▪ Portus ▪ Artifactory (JFrog)

Source: Gartner (January 2026)

- **Ensuring artifact registries are OCI-conformant:** Require all container registries to be OCI-compliant, as the OCI standard ensures consistent formats and interoperability. Ensure a standards-first approach that reduces reliance on proprietary solutions and simplifies vendor migration. It also ensures portability for various artifacts, including container images, AI models, parameters, and templates.

- **Investing in proximity-based automated workflows:** Implement automated workflows for building and pushing images to registries close to the execution platforms. Optimizing the image-pulling process requires placing the registry as close to the platform consumers as possible. Maintaining registries near platforms can also significantly speed up image transfers and cache utilization. This streamlined approach is key because container image registries are part of the infrastructure where underlying vendor-specific implementations can inadvertently deepen dependence.

This research note is restricted to the personal use of .

- **Selecting registries with native Governance features (replication and syncing):** When images need to be required across different infrastructures (e.g., in multicloud/hybrid environments), organizations should prioritize selecting artifact registries that support replication and syncing. Modern registries offer advanced governance features such as replication/syncing with external registries, pull-through proxy cache, and retention policies. These are highly recommended practices for managing dependencies and maintaining consistent image availability. ORAS (OCI Registry As Storage), Skopeo, and Crane are open-source initiatives that implement replication and synchronization capabilities.

- **Leverage distribution technologies for ORAS:** As an alternative solution for managing images across different infrastructures, use distribution technologies like ORAS (OCI Registry As Storage) to easily distribute artifacts across OCI registries. ORAS is a tool that practitioners can use to distribute artifacts, including significant artifacts such as AI models, across OCI-compliant registries. Initiatives like ORAS allow organizations to leverage S3-compatible object storage as back-end storage for OCI images, enabling parallel chunks' transfer and significantly improving transfer speed for significant artifacts.

1.6. Fleet Management

Managing container platforms at scale necessitates specialized tooling and services to address the complexities of modern infrastructure. Effective fleet management extends beyond offering a single pane of glass; it encompasses multicluster management across hybrid and multicloud environments, strategic workload placement, unified governance, and network traffic management. These capabilities are essential for maintaining operational efficiency, ensuring compliance, and optimizing resource utilization.

Minimize fleet management dependence by:

- **Prioritizing unified interfaces and feature parity:** Select fleet management platforms that provide unified APIs and consistent feature sets across all supported environments. This approach streamlines operations, simplifies integration, and reduces the risk of fragmentation, ensuring seamless management regardless of underlying infrastructure or cloud provider (see Comparing Approaches for Kubernetes Cluster Fleet Management).

This research note is restricted to the personal use of .

- **Leveraging open source and distribution-neutral solutions:** Opt for management capabilities backed by robust open-source initiatives and distribution-neutral tools such as Spectro Cloud, D2IQ, Rancher, k0rdent, Taikun, Northflank, and Portainer. These platforms foster interoperability, community-driven innovation, and reduce vendor lock-in, granting organizations greater flexibility and control over their container environments.

- **Adopting open provisioning and workload placement standards:** Favor open platform provisioning methods, such as ClusterAPI, or declarative () tools like Terraform (TF) and OpenTofu (OTF). Promote open workload delivery and placement mechanisms, including tools (ArgoCD, Flux, Fleet), OCM, Karmada, KCP, and Liqo, to enable automated, reproducible, and portable deployments across diverse infrastructures.

- **Evaluating inter-platform networking initiatives as needed:** When multicloud or hybrid architectures require cross-cluster connectivity, assess inter-platform networking solutions such as Skupper, Submariner, and Kubeslice. These technologies facilitate secure, reliable communication and workload mobility, reducing operational dependence on proprietary networking implementations.

Table 2 summarizes the main initiatives associated with infrastructure components that help practitioners to promote vendor neutrality.

This research note is restricted to the personal use of .

Table 2: Container Infrastructure Components

(Enlarged table in Appendix)

Component	Description	Standards	OSS Initiatives/Community Projects
Computing	The underlying compute resources (CPU, GPU, specialized accelerators) must support open standards to prevent workloads from being tied to proprietary APIs or hardware. Minimizing dependence involves supporting multi-arch images (e.g., x86_64 and ARM).	▪ OCI (for multi-arch images) ▪ OpenCL	AMD's ROCm, Intel's oneAPI, HAMi (Heterogeneous AI Computing Virtualization Runtime), Volcano, TensorFusion, Dynamic Resource Allocation (DRA).
Networking	Manages North-South (ingress/traffic routing) and East-West (inter-pod) container traffic using standardized interfaces.	▪ CNI (Container Network Interface) ▪ Gateway API ▪ GAMMA initiative (encompasses meshing technologies).	Cilium, Calico, NetworkPolicy API (K8s native specs), Istio, Kuma, Linkerd.
Storage	Provides persistent storage for stateful applications, which is challenging due to diverse access requirements (Block, File, Object) and performance characteristics.	▪ CSI (Container Storage Interface).	CEPH, Longhorn, OpenEBS features like dynamic provisioning, Snapshots.
Runtime	Manages the container life cycle by interfacing between the orchestration platform (like Kubernetes) and the container execution environment.	▪ CRI (Container Runtime Interface) ▪ OCI (Open Container Initiative) specifications.	containerd, CRI-O, Docker, specialized runtimes/extensions like Kuasar, gisor, Firecracker.
Artifact Registries	Centralized repositories for storing and distributing reusable software components, ensuring image portability.	▪ OCI (Open Container Initiative) standard.	Harbor, Quay, ORAS (for artifact distribution across OCI registries).
Fleet Management	Managing multiple container platforms and clusters at scale, focusing on multi-cluster life cycle management and unified governance across distributed environments.	▪ (Relies heavily on declarative IaC/GitOps practices).	Cluster API, KCP, K0rdent, GitOps tools (ArgoCD, Flux, Fleet).

Source: Gartner (January 2026)

Container Runtime

Container runtime refers to the layer between the container infrastructure components and the containerized workloads. It acts as an application support layer, mediating the interaction between the application execution environment and the underlying infrastructure.

This research note is restricted to the personal use of .

While runtime components provide standardized interfaces and abstraction layers within containerized environments, their mere presence does not ensure application portability. Achieving true portability requires that applications be architected to leverage these runtime services, adhering to best practices such as stateless design, externalized configuration, and decoupled dependencies. Without thoughtful application design that aligns with runtime capabilities, teams may still encounter challenges when migrating workloads across different platforms or cloud providers, limiting the benefits of vendor-neutral infrastructure.

Figure 5: Runtime Standards and Patterns for Containers

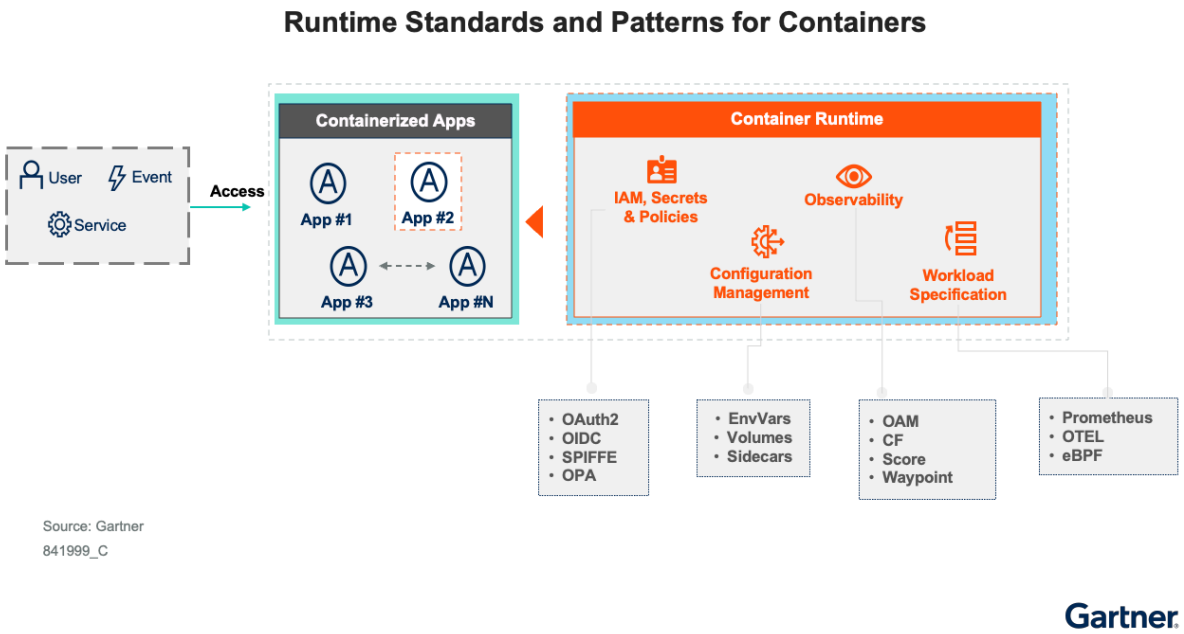


Figure 5 depicts the key container runtime components that significantly influence vendor neutrality. Each plays a crucial role in ensuring containerized applications remain portable, interoperable, and free from proprietary constraints.

2.1. Configuration Management

This research note is restricted to the personal use of .

Container platforms must be able to inject configuration parameters into the container execution space. K8s addresses this need by offering ConfigMap Resources, which can be provided via environment variables or mounted from external sources. However, other enterprise container platforms adopt different approaches, based on imperative methods, envfiles, or sidecars. While applications can be instrumented to obtain configuration parameters directly from back-end platforms, doing so requires leveraging an application runtime (e.g., NATS, DAPR) to neutralize discrepancies between platforms. Implementing consistent methods across different container platforms is part of a broader strategy to adopt open industry specifications and standardize the container runtime layer to mitigate vendor dependence.

Minimize configuration management dependence by:

- **Mapping application requirements for dynamic configuration:** Conduct a comprehensive assessment of each application's configuration needs, including environment variables, configuration files, secrets, and mounted volumes.
- **Evaluating platform support for configuration methods:** Analyze the capabilities and limitations of your chosen container orchestrators (e.g., Kubernetes, ECS, Nomad) to ensure they can accommodate your application's configuration needs. Recognize that not all platforms natively support every configuration method (e.g., secret injection or volume mounting), and plan appropriate workarounds or extensions where necessary.
- **Establishing a single source of truth for configuration parameters:** Choose a centralized, platform-agnostic parameter store, such as HashiCorp Consul, to manage all configuration data and secrets. Implement automated Infrastructure as Code (IaC) workflows to provision, update, and audit configuration parameters, ensuring consistency, traceability, and compliance across environments.
- **Standardizing the configuration injection process:** Define and enforce a standardized approach for injecting configuration data into applications, whether during the continuous deployment (CD) pipeline or at runtime using platform-native features. This standardization reduces operational complexity, mitigates risks of misconfiguration, and enhances portability by decoupling configuration management from specific vendors or platforms.

2.2. Container Observability

This research note is restricted to the personal use of .

Container orchestrators must be capable of collecting, potentially transforming, and forwarding crucial observability signals, such as logs, metrics, and traces, to backend services for storing and subsequent analysis. Although proprietary methods exist in the industry, the cloud-native landscape increasingly favors widely accepted open standards to minimize vendor dependence. Prometheus is recognized as the standard for exposing and collecting metrics for monitoring. Complementing this, (OTEL) provides a vendor-neutral standard for generating, collecting, and managing telemetry data, including logs, metrics, and traces. These open standards are vital for comprehensive IT operations and observability. Crucially, there is typically no impediment preventing commercial stacks from supporting Prometheus or OTEL, and most currently do so.

Minimize container observability dependence by:

- **Selecting observability backends adherent to open standards:** Choose and backend services that fully support open standards such as OpenTelemetry, Prometheus, and other observability specifications (e.g., Prometheus Exposition Format, CloudEvents). It ensures interoperability, simplifies integration across diverse environments, and reduces the risk of vendor lock-in, allowing organizations to switch or scale observability solutions as needed.
- **Ensuring observability signal collection conforms to open standards:** Instrument applications and infrastructure to generate metrics, logs, and traces in formats compliant with open standards (e.g., OTEL), and leverage -based observability for deep, low-overhead telemetry from the kernel and network layers. Forward these signals to backend services using standardized protocols, enabling consistent monitoring and analysis across observability stacks
- **Promoting open standards adoption among application teams:** Encourage development teams to utilize open-source libraries and frameworks for observability instrumentation, such as OpenTelemetry SDKs. Provide training and resources to ensure teams understand the benefits of open standards and how to implement them effectively in their workflows.
- **Facilitating portability and future-ready observability data:** By standardizing on open formats and protocols, organizations ensure that observability data remains portable and accessible, even as backend technologies evolve. This approach supports long-term scalability, easier tool migration, and alignment with industry best practices for cloud-native observability.

2.3. IAM, Secrets & Policies

This research note is restricted to the personal use of .

Container platforms must securely manage and inject sensitive data, such as passwords and API keys, using secret management features. External secret managers can be employed to enhance control, auditing, and secure delivery of this sensitive data to workloads. Furthermore, containerized workloads achieve secure, dynamic authentication and authorization by implementing identity solutions, including Service Accounts (SA) or pod identity. For governance, many platforms offer policy enforcement capabilities, often implemented through CI/CD pipelines or admission controllers, though users can also enable these features. These security and governance measures address the management of identity, secrets, and policies within the container runtime ecosystem.

Minimize the dependence on vendor-specific IAM, secret and policy management by:

- **Storing secrets in vendor-neutral backend management solutions:** Utilize centralized secret management platforms such as HashiCorp Vault or managed solutions but ensure secrets are injected into execution environments or platforms that support portability. Evaluate the injection method case-by-case basis to maintain flexibility and avoid complex dependencies on platform-specific secret management features.
- **Adopting industry-standard protocols for identity federation and authentication:** Implement protocols like OAuth2, OIDC, and for identity federation and workload authentication, enabling portable, platform-independent identity management. Leverage tools such as KeyCloak, Teleport, CyberArk, and StrongDM to maintain interoperability across cloud providers and on-premises environments, reducing reliance on proprietary IAM solutions.
- **Shifting policy enforcement to ci/cd workflows:** Integrate policy validation and enforcement into CI/CD pipelines using open-source or vendor-neutral tools, rather than relying solely on platform-integrated policy engines (e.g., Azure Policy, AWS Config). This approach ensures consistent policy application across environments and enhances portability, compliance, and auditability.
- **Standardizing secret and policy management practices:** Establish organizationwide guidelines for secret storage, retrieval, and policy enforcement that prioritize open standards and interoperability. Regularly review and update practices to align with evolving industry standards and security best practices, ensuring long-term vendor neutrality and operational resilience.

2.4. Workload Specification

This research note is restricted to the personal use of .

Currently, there is no single and universal workload specification for containers adopted across all platforms and ecosystems. Initiatives promoting universal specifications, such as HashiCorp Waypoint, CloudFoundry, and Score.dev, have not achieved widespread adoption. K8s-oriented efforts, including OAM (implemented by Crossplane and KubeVela) and the defunct CNAB, have emerged with this same goal, but equally not widely adopted. Consequently, practitioners must rely on platform-specific formats, such as K8s Kubernetes manifests, or on platform-supported formats.

To minimize this dependency, teams must adopt community initiatives (whenever possible) and prioritize declarative approaches. Leveraging templating tools (e.g., Helm) and patching tools (e.g., Kustomize) helps bridge gaps across varying definitions, ultimately enhancing workload portability and managing long-term dependency risks.

Minimize dependency on workload specification by:

- **Utilizing templating mechanisms for kubernetes:** Adopt templating tools such as Helm or similar solutions to abstract and parameterize Kubernetes manifests. These mechanisms enable reusable, modular workload definitions, simplifying deployment processes and enhancing portability across different clusters and environments.
- **Assessing kubernetes controllers for cross-platform orchestration**: For advanced scenarios, evaluate Kubernetes controllers and operators that extend orchestration capabilities to other container platforms, such as Crossplane AWS ECS Provider or Crossplane GCP CloudRun Provider. Integrating these controllers enables unified workload management across multiple cloud providers, minimizing reliance on native workload specifications.
- **Leveraging overlay and patching tools for multi-format rendering (with caution):** Use tools like Kustomize, YTT, or YQ to dynamically render and adapt workload definitions across different platforms and formats. While these tools offer flexibility, they can introduce complexity and maintenance challenges, so their use should be carefully considered and limited to scenarios where more robust solutions are not feasible.

Table 3 summarizes the main initiatives associated with runtime components that help practitioners to promote vendor neutrality.

This research note is restricted to the personal use of .

Table 3: Container Runtime Components

(Enlarged table in Appendix)

Component	Description	Standards	Open Source Initiatives/Community Projects
Configuration Management	Governing how configuration parameters are securely injected and consistently managed within container environments.	▪ EnvVars, Volumes ▪ ConfigMap Resources (K8s native).	Consult, NATS, DAPR, IaC tooling (e.g., Terraform).
Observability	Collecting, transforming, and forwarding logs, metrics, and traces to back-end services for analysis.	▪ OTEL (OpenTelemetry) (vendor-neutral standard for telemetry data) ▪ Prometheus (standardized way metrics are exposed/collected).	eBPF (for data collection), Prometheus, Grafana Beyla, Pixie.
IAM, Secrets & Policies	Securely managing sensitive data (secrets) and providing portable authentication/authorization for containerized workloads. Policy enforcement is often achieved through admission controllers or CI/CD pipelines.	▪ OAuth2 ▪ OIDC (OpenID Connect) ▪ SPIFFE ▪ OPA (Open Policy Agent)	HashiCorp Vault, KeyCloak, admission controllers (e.g., Kyverno, Gatekeeper).
Workload Specification	Defining application workloads using platform-agnostic packaging formats helps mitigate reliance on specific formats, such as K8s manifests.	▪ OAM (Open Application Model), ▪ CF (CloudFoundry) ▪ Score.	HashiCorp Waypoint, KubeVela, Crossplane, Helm (templating), Kustomize (patching/overlay)

Source: Gartner (January 2026)

This research note is restricted to the personal use of .

Risks and Pitfalls

- **Increased operational complexity:** Minimizing vendor dependence can lead to increased operational complexity and technical overhead. This is because managing infrastructure stacks and abstractions across different environments requires significant effort. Pursuing complete vendor independence often results in unnecessary complexity and higher costs, requiring heavy investment in systems engineering.

- **Incurring the "portability tax":** Achieving vendor neutrality often comes with a "portability tax" — increased costs and high engineering overhead. To achieve true portability, organizations must invest heavily in developing internal expertise to build and maintain vendor-neutral solutions.

- **Sacrificing specialized features**: Pursuing vendor neutrality may force applications to adopt a "least common denominator" approach, sacrificing access to innovative, specialized features and integrated support offered by proprietary vendor capabilities and managed services. In some cases, access to proprietary functions outweighs the benefits of platform neutrality.

- **Substituting one lock-in for another:** Teams risk replacing one form of vendor dependence with another, potentially leading to lock-in to the K8s abstraction layer or commercial distribution software. They may also become dependent on complex custom integration ("glue") between open-source components.

- **Relying on Emerging Vendors**: Relying on small or unproven vendors to achieve neutrality often face higher operational and security risks. Mitigate this risk by conducting due diligence, prioritizing those with proven track records, financial stability, and mature technologies before adoption.

- **Misalignment with strategic objectives:** Organizations must weigh long-term flexibility against immediate ease of deployment and operational burden. If extensive portability is not a primary strategic objective, it may be more practical to focus team skill sets on proprietary technologies to prioritize operational efficiency.

This research note is restricted to the personal use of .

Acronym Key and Glossary Terms

OCI (Open Container Initiative)	An open governance structure that defines industry standards for container image formats and runtimes, ensuring interoperability and portability across container platforms.
CSI (Container Storage Interface)	A standard API for exposing and managing storage systems in container orchestration platforms (e.g., Kubernetes), supporting dynamic provisioning and life cycle management of persistent storage.
CRI (Container Runtime Interface)	A Kubernetes-specific API that defines how the orchestration platform communicates with different container runtimes, enabling pluggable runtime support.
CNI (Container Network Interface)	A specification for configuring network interfaces in Linux containers, enabling networking for containers and supporting a variety of networking solutions and plugins.
Gateway API	A Kubernetes API standard for advanced traffic management, enabling more expressive and extensible configuration of networking resources such as load balancers, ingress, and service mesh integration.
GAMMA (Gateway API for Mesh Management and Administration)	An extension of the Gateway API, focused on standardizing service mesh management and administration within Kubernetes environments.
Cluster API	An open-source Kubernetes project and specification for declarative management of Kubernetes cluster life cycle, including creation, configuration, and scaling of clusters across multiple infrastructures.
OpenCL (Open Container Library)	Not to be confused with the Open Computing Language, OpenCL in the container context refers to libraries and specifications for managing container images and runtimes, though this term is rarely used in mainstream container orchestration.
HAMi (Horizontal Application Migration Initiative)	An emerging specification aimed at standardizing the migration of containerized applications across heterogeneous environments, promoting portability and minimizing vendor lock-in.

This research note is restricted to the personal use of .

Evidence

[1] Cilium Annual Report 2025: A Decade of Cloud Native Networking, CNCF.io

[2] Gateway API: Introduction, Gateway API

[3] The GAMMA Initiative (Gateway API for Service Mesh), Gateway API

Recommended by the Author

Some documents may not be available as part of your current Gartner subscription.

Use Cloud-Native Architecture to Modernize Your Applications

Optimize AI Workloads With Containers and Kubernetes

Deploying Containers and Kubernetes at the Edge

Best Practices for Container Release and Life Cycle Management

Reference Architecture Brief: AI Infrastructure With Kubernetes

Adapting Kubernetes for AI Inference and Serving Workloads

© 2026 Gartner, Inc. and/or its affiliates. All rights reserved. Gartner is a registered trademark of Gartner, Inc. and its affiliates. This publication may not be reproduced or distributed in any form without Gartner's prior written permission. It consists of the opinions of Gartner's Business and Technology Insights Organization, which should not be construed as statements of fact. While the information contained in this publication has been obtained from sources believed to be reliable, Gartner disclaims all warranties as to the accuracy, completeness or adequacy of such information. Although Gartner insights may address legal and financial issues, Gartner does not provide legal or investment advice and its insights should not be construed or used as such. Your access and use of this publication are governed by Gartner's Usage Policy. Gartner prides itself on its reputation for independence and objectivity. Its insights is produced independently by its Business and Technology Insights Organization without input or influence from any third party. For further information, see " Guiding Principles on Independence and Objectivity." Gartner insights may not be used as input into or for the training or development of generative artificial intelligence, machine learning, algorithms, software, or related technologies.

This research note is restricted to the personal use of .

Table 1: Container Registry Alternatives

Managed Container Registries	Self-hosted Container Registries
Amazon Elastic Container Registry (ECR)	Harbor
Google Artifact Registry (formerly Google Container Registry, GCR)	Docker Distribution (Docker Registry)
Azure Container Registry (ACR)	Quay
GitHub Container Registry	Nexus Repository OSS
GitLab Container Registry	GitLab Container Registry
Quay.io	Portus
Docker Hub	Artifactory (JFrog)
Oracle Cloud Container Registry	
IBM Cloud Container Registry	

Source: Gartner (January 2026)

This research note is restricted to the personal use of .

Table 2: Container Infrastructure Components

Component	Description	Standards	OSS Initiatives/Community Projects
Computing	The underlying compute resources (CPU, GPU, specialized accelerators) must support open standards to prevent workloads from being tied to proprietary APIs or hardware. Minimizing dependence involves supporting multi-arch images (e.g., x86_64 and ARM).	■ OCI (for multi-arch images) ■ OpenCL	AMD's ROCm, Intel's oneAPI, HAMi (Heterogeneous AI Computing Virtualization Runtime), Volcano, TensorFusion, Dynamic Resource Allocation (DRA).
Networking	Manages North-South (ingress/traffic routing) and East-West (inter-pod) container traffic using standardized interfaces.	■ CNI (Container Network Interface) ■ Gateway API ■ GAMMA initiative (encompasses meshing technologies).	Cilium, Calico, NetworkPolicy API (K8s native specs), Istio, Kuma, Linkerd.
Storage	Provides persistent storage for stateful applications, which is challenging due to diverse access requirements (Block, File, Object) and performance characteristics.	■ CSI (Container Storage Interface).	CEPH, Longhorn, OpenEBS features like dynamic provisioning, Snapshots.
Runtime	Manages the container life cycle by interfacing between the orchestration	■ CRI (Container Runtime Interface)	containerd, CRI-O, Docker, specialized runtimes/extensions like Kuasar,

This research note is restricted to the personal use of .

	platform (like Kubernetes) and the container execution environment.	■ OCI (Open Container Initiative) specifications.	gisor, Firecracker.
Artifact Registries	Centralized repositories for storing and distributing reusable software components, ensuring image portability.	■ OCI (Open Container Initiative) standard.	Harbor, Quay, ORAS (for artifact distribution across OCI registries).
Fleet Management	Managing multiple container platforms and clusters at scale, focusing on multi-cluster life cycle management and unified governance across distributed environments.	■ (Relies heavily on declarative IaC/GitOps practices).	Cluster API, KCP, K0rdent, GitOps tools (ArgoCD, Flux, Fleet).

Source: Gartner (January 2026)

This research note is restricted to the personal use of .

Table 3: Container Runtime Components

Component	Description	Standards	Open Source Initiatives/Community Projects
Configuration Management	Governing how configuration parameters are securely injected and consistently managed within container environments.	▪ EnvVars, Volumes ▪ ConfigMap Resources (K8s native).	Consult, NATS, DAPR, IaC tooling (e.g., Terraform).
Observability	Collecting, transforming, and forwarding logs, metrics, and traces to back-end services for analysis.	▪ OTEL (OpenTelemetry) (vendor-neutral standard for telemetry data) ▪ Prometheus (standardized way metrics are exposed/collected).	eBPF (for data collection), Prometheus, Grafana Beyla, Pixie.
IAM, Secrets & Policies	Securely managing sensitive data (secrets) and providing portable authentication/authorization for containerized workloads. Policy enforcement is often achieved through admission controllers or CI/CD pipelines.	▪ OAuth2 ▪ OIDC (OpenID Connect) ▪ SPIFFE ▪ OPA (Open Policy Agent)	HashiCorp Vault, KeyCloak, admission controllers (e.g., Kyverno, Gatekeeper).
Workload Specification	Defining application workloads using platform-agnostic packaging formats	▪ OAM (Open Application Model),	HashiCorp Waypoint, KubeVela, Crossplane, Helm (templating),

This research note is restricted to the personal use of .

	helps mitigate reliance on specific formats, such as K8s manifests.	■ CF (CloudFoundry) ■ Score.	Kustomize (patching/overlay)

Source: Gartner (January 2026)

This research note is restricted to the personal use of .